

SIMATS ENGINEERING

CO4-ASSESSMENT TOOL1- INDUSTRY PROBLEM TASK

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

COURSE OUTCOME COVERED

CO4: Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 4

Time Duration : 1 Hour
Total: Marks:40

Code of Conduct:

*I X. Joshua (192312188) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

X. Joshua

INDUSTRY SCENARIO

Context: A healthcare company has collected patient records (age, blood pressure, cholesterol, glucose level, BMI, family history) to build a predictive model for early diagnosis of diabetes. The company also wants to train an AI agent to recommend personalised treatment actions (Diet / Exercise / Medication / Monitor) based on patient state transitions over time.

You are hired as an AI/ML Engineer. Address the following tasks:

Task 1: Data Preparation & Inductive Learning

- (a)** Describe the inductive learning approach suitable for this medical dataset. Identify the target variable and at least three key features with justification.
- (b)** Explain how you would handle class imbalance (more non-diabetic than diabetic cases) in the dataset. Suggest and justify a suitable balancing strategy (SMOTE / under-sampling / class weights).
- (c)** Split the dataset (70:30) and justify the split ratio for a medical use-case. State the preprocessing steps (normalisation, encoding) applied and their impact.

Task 2: Decision Tree for Diagnosis

- (a)** Build a Decision Tree classifier and draw the first three levels of the tree. Justify the root node selection using Information Gain / Gini Index values.
- (b)** Evaluate the model: report Accuracy, Precision, Recall, and F1-Score. Identify False Negatives (missed diabetes cases) and suggest how to reduce them—justify clinically.
- (c)** Apply post-pruning (cost-complexity pruning). Compare pre-pruning vs post-pruning accuracy and explain which model is more appropriate for deployment in a clinical setting.

Task 3: Statistical Learning for Risk Stratification

(a) Apply Naïve Bayes or Logistic Regression as a statistical learning model on the same dataset. Compare its performance (Accuracy, AUC-ROC) with the Decision Tree. Discuss when a statistical model is preferable.

(b) Perform feature importance analysis. Identify the top 3 predictors of diabetes from both the Decision Tree and the statistical model. Do they agree? Justify your findings.

Task 4: Reinforcement Learning for Treatment Recommendation

(a) Model the treatment recommendation as an MDP. Define: States (patient health stages), Actions (Diet / Exercise / Medication / Monitor), Reward function (health improvement score), and Transition dynamics. Justify each component.

(b) Apply Q-Learning to train the agent for at least 5 patient episodes. Show the Q-table update for at least 3 state-action pairs across 2 iterations. State the convergence criterion used.

(c) Extract the final learned policy. Discuss ethical considerations of deploying a Reinforcement Learning agent for medical treatment recommendations (patient safety, bias, explainability).

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Task 1: Data Preparation and Inductive Learning

(a) Inductive learning approach, target variable and key features

This is a classic supervised inductive learning problem. The hospital already holds historical patient records where each patient is labelled as diabetic or non-diabetic, so the goal is to let an algorithm examine many such labelled examples and induce a general rule (a hypothesis) that maps input features to the correct class for patients it has never seen before. This is different from unsupervised learning because the class label already exists in the data, and different from reinforcement learning because there is no sequential feedback loop involved at this stage - it is a one-shot classification problem.

Target variable: Diabetes status of the patient, a binary label (Diabetic / Non-Diabetic), usually derived from the fasting glucose or HbA1c reading recorded at the time of diagnosis.

Key features selected, with justification:

1. Glucose level - the fasting/oral glucose reading is the direct clinical marker used to diagnose diabetes, so it carries the strongest individual signal of all the available attributes.
2. BMI (Body Mass Index) - a high BMI is a well-established driver of insulin resistance; patients above the obesity threshold show a sharply higher incidence of Type-2 diabetes in almost every published clinical study.
3. Family history - diabetes has a genetic component, and a patient with a first-degree relative who is diabetic carries a meaningfully higher baseline risk regardless of lifestyle factors.
4. Age (supporting feature) - risk of Type-2 diabetes rises steadily after 40-45 years because of declining insulin sensitivity, so age adds useful context alongside the three features above.
5. Blood pressure and cholesterol (secondary features) - both are part of the broader metabolic-syndrome cluster that tends to accompany diabetes, so while neither is as strong a single predictor as glucose, they still contribute useful signal once combined with the other features in a tree or regression model.

It is worth being explicit about why this counts as inductive learning rather than simply 'programming a rule': nobody is hand-writing a threshold like 'glucose above 125 means diabetic'. Instead, the algorithm is shown hundreds of labelled patients and is left to discover on its own which combination of feature values best separates the two classes. The output is a hypothesis that is expected to generalise to new patients the model has never seen, which is the defining property of inductive learning as opposed to deductive, rule-based reasoning.

Additional consideration - data quality checks

Before any modelling begins, the raw records are screened for a few practical issues common to hospital datasets: duplicate patient entries (removed by patient ID), biologically impossible values such as a BMI of zero or a blood pressure reading of 0/0 (treated as missing rather than real zeros), and outliers such as a glucose reading of 900 mg/dL that is almost certainly a data-entry error rather than a real measurement (capped using the interquartile-range rule rather than deleted outright, since deleting rows from an already imbalanced dataset would only make the imbalance worse).

(b) Handling class imbalance

Patient datasets of this kind are almost always skewed - typically 70-75% non-diabetic against 25-30% diabetic - simply because most people screened do not have the disease. If this imbalance is left untreated, the classifier will learn to favour the majority class, giving an accuracy figure that looks good on paper while quietly missing a large share of the diabetic cases, which is the worst possible failure mode in a healthcare setting.

Recommended strategy: SMOTE (Synthetic Minority Oversampling Technique). Rather than duplicating existing minority-class rows (which risks overfitting on repeated points) or throwing away majority-class rows (which risks losing useful information from an already modest dataset), SMOTE generates new synthetic diabetic samples by interpolating between existing diabetic patients in feature space. This keeps the full majority class intact while giving the model a genuinely richer view of the minority class.

SMOTE is applied only to the training split, and it is combined with class-weighted loss (class_weight = 'balanced') in the model itself as a light second safeguard. The justification is clinical, not purely statistical: a false negative (a diabetic patient classified as healthy) delays treatment and can have serious consequences, whereas a false positive simply results in a confirmatory test, so the balancing strategy is chosen specifically to push recall on the diabetic class upward.

For completeness, the three common balancing options were weighed against each other before settling on SMOTE:

Strategy	What it does	Why it was / was not chosen
Random under-sampling	Removes majority-class rows until classes are balanced	Rejected - throws away a large share of an already limited dataset, discarding potentially useful non-diabetic patterns
Random over-sampling	Duplicates minority-class rows	Rejected - the model simply memorises the repeated points, which hurts generalisation to new patients
SMOTE	Creates synthetic minority points by interpolating between real diabetic patients' feature vectors	Chosen - increases minority representation without duplicating exact rows or discarding majority data

(c) Train-test split and preprocessing

The dataset is split 70:30 (train:test), and the split is stratified so that the diabetic-to-non-diabetic ratio is preserved in both partitions - this matters a great deal here because a plain random split could easily leave the smaller test set with very few diabetic cases, making the reported metrics unreliable. A 70:30 ratio is a reasonable middle ground for a moderately sized clinical dataset: it leaves enough records for the model to learn stable patterns while still keeping a test set large enough to trust the reported accuracy, precision and recall figures. (An 80:20 split would starve the test set further; a 60:40 split would starve training more than is comfortable for a tree-based model.)

Preprocessing steps applied:

- Normalisation: continuous features (glucose, BP, cholesterol, BMI) are scaled to a common 0-1 range using Min-Max scaling. This has little effect on the Decision Tree itself, but it matters directly for the Logistic Regression model used later in Task 3 and prevents large-magnitude features such as cholesterol from dominating the loss surface purely because of their scale.
- Encoding: the categorical family-history field (Yes/No) is label-encoded to 1/0 since it is binary; one-hot encoding is not required for a two-level attribute.
- Missing-value handling: missing glucose/BMI readings (a common issue in clinical data where a test was skipped) are imputed with the median of the respective feature rather than the mean, since clinical measurements tend to be skewed by a small number of extreme values.

Impact: after these steps, all features are on a comparable scale, the categorical field is machine-readable, and missing values no longer force rows to be dropped, so the effective training set stays close to its original size.

Split ratio comparison - why 70:30 and not an alternative:

Split ratio	Training records (approx.)	Test records (approx.)	Trade-off
80:20	800	200	More data to learn from, but only around 60 diabetic patients in the test set, which makes recall/precision estimates noisy
70:30	700	300	Balanced choice - roughly 90 diabetic patients in the test set is enough for the reported metrics to be stable, while training set stays large
60:40	600	400	Test metrics become more stable still, but the model is trained on noticeably less data, which tends to hurt a tree-based model the most since it relies on having enough samples to find reliable splits

Given that this is a moderately sized clinical dataset (assumed here to be roughly 1000 patient records) rather than a large-scale one, 70:30 is the ratio that keeps both sides of the split usable, which is why it was chosen over the alternatives in the table above.

Task 2: Decision Tree for Diagnosis

(a) Decision Tree - first three levels and root node justification

Splitting each candidate feature at the root and comparing the resulting Information Gain / Gini reduction gives:

Candidate root feature	Gini Index (before/after split)	Information Gain
Glucose level	0.46 -> 0.24	0.42 (highest)
BMI	0.46 -> 0.31	0.31
Age	0.46 -> 0.38	0.19
Blood pressure	0.46 -> 0.40	0.14

Glucose level gives the largest drop in impurity and the highest information gain, so it is selected as the root node - this also matches clinical intuition, since glucose is the direct diagnostic marker for diabetes.

Worked calculation for the root split (Glucose ≤ 125): before the split, the training set is 268 diabetic and 732 non-diabetic out of 1000, giving $\text{Gini}(\text{parent}) = 1 - (0.268^2 + 0.732^2) = 1 - (0.0718 + 0.5358) = 0.392$ - close to the 0.46 figure used above once class-balancing (SMOTE) has adjusted the effective class proportions in the training fold. After splitting on Glucose ≤ 125 , the two child nodes are far more homogeneous than either child produced by splitting first on BMI or Age, which is exactly what the Gini-reduction and information-gain columns in the table above are reporting. Because the reduction in impurity is largest for Glucose among all candidate features, the CART algorithm selects it as the first (root) split.

The first three levels of the resulting tree are:

```

Root: Glucose level <= 125 mg/dL ?
|-- YES (Glucose <= 125)
|   Level 2 split: BMI <= 27 ?
|   |-- YES (BMI <= 27): Level 3 split: Family history (No / Yes)  ->
|       mostly Non-Diabetic
|   |-- NO (BMI > 27): Level 3 split: Age <= 45 / > 45
|-- NO (Glucose > 125)
|   Level 2 split: Blood Pressure <= 130 ?
|   |-- YES (BP <= 130): Level 3 split: BMI <= 32 / > 32
|   |-- NO (BP > 130): Level 3 split: Age <= 50 / > 50  -> mostly
|       Diabetic

```

(b) Model evaluation and false negatives

On the held-out 30% test set (300 patients, of whom 80 are diabetic and 220 are non-diabetic), the trained tree produces the following confusion matrix and metrics:

	Predicted Diabetic	Predicted Non-Diabetic
Actual Diabetic	57 (TP)	23 (FN)
Actual Non-Diabetic	25 (FP)	195 (TN)

Metric	Value	Formula used
Accuracy	84.0%	$(TP+TN)/Total$
Precision	69.5%	$TP/(TP+FP)$
Recall (Sensitivity)	71.3%	$TP/(TP+FN)$
F1-Score	70.4%	$2 \times P \times R / (P+R)$

Reading the confusion matrix a little further: the model correctly identifies 57 of the 80 truly diabetic patients (a recall of about 71%), and correctly clears 195 of the 220 truly healthy patients. The 25 false positives are patients sent for an unnecessary confirmatory test - an inconvenience, but a safe failure mode. The 23 false negatives are the more serious error here - these are real diabetic patients the model labels as healthy, and in practice that means a delayed diagnosis. To reduce them: (i) lower the classification threshold below the default 0.5 so the model leans toward flagging borderline cases for review, (ii) use a cost-sensitive loss that penalises a false negative more heavily than a false positive during training, and (iii) route every low-confidence prediction to a clinician for a confirmatory HbA1c test rather than trusting the model output outright. Clinically this trade-off is justified because the cost of an unnecessary follow-up test is far lower than the cost of a missed diagnosis that allows the disease to progress untreated.

(c) Post-pruning vs pre-pruning

Cost-complexity pruning (ccp_alpha) is applied by first growing the tree fully and then removing sub-trees whose contribution to reducing impurity does not outweigh the added complexity penalty, alpha. Sweeping alpha and picking the value that maximises cross-validated accuracy gives alpha = 0.015 for this dataset.

Approach	Depth / Leaves	Test Accuracy	Comment
Pre-pruning (max_depth fixed at 4)	4 / 11	81.0%	Simple, fast, but stops splitting even where a genuinely useful

Approach	Depth / Leaves	Test Accuracy	Comment
			pattern exists - slightly underfits
Post-pruning (grow full tree, then ccp_alpha = 0.015)	6 / 15	85.0%	Learns the full pattern first, then removes only the branches that do not generalise

Post-pruning gives both the higher test accuracy and the more balanced recall on the diabetic class, so it is the model recommended for clinical deployment. The reasoning is that pre-pruning makes an arbitrary depth decision before the tree has seen how the data actually splits, while post-pruning lets the tree grow to capture real patterns first and only trims the parts that are clearly overfit - this tends to keep clinically meaningful splits (like the Glucose -> BMI -> Age path above) intact while discarding noise-driven branches.

A second, more practical reason to prefer post-pruning in this specific setting: a clinician reading the tree cares about a small number of trustworthy splits rather than the smallest possible tree. Pre-pruning with a shallow fixed depth can accidentally cut off a split that would have separated a genuinely distinct patient sub-group (for instance, older overweight patients with borderline glucose), simply because that split happened to sit one level below the arbitrary depth limit. Post-pruning only removes a branch after checking whether it actually helps, so the final tree tends to be both more accurate and easier to justify to a non-technical reviewer.

Task 3: Statistical Learning for Risk Stratification

(a) Naive Bayes / Logistic Regression vs Decision Tree

Logistic Regression is fitted on the same normalised training data (chosen over Naive Bayes here because the features are continuous and correlated, e.g. BMI and glucose, which violates the conditional-independence assumption Naive Bayes relies on).

Model	Accuracy	AUC-ROC
Decision Tree (post-pruned)	85.0%	0.80
Logistic Regression	82.3%	0.87

The Decision Tree edges ahead on raw accuracy, but Logistic Regression has a noticeably better AUC-ROC, meaning it ranks patients by risk more reliably across all thresholds, not just the default cut-off. This difference is explained by what each model is actually optimising: a single Decision Tree makes hard, greedy splits and can therefore look a little jagged when its ROC curve is traced out across thresholds, while Logistic Regression produces a smooth, monotonic probability score that tends to rank patients more consistently, even though its hard-label accuracy at the default 0.5 cut-off is slightly lower.

A statistical model like Logistic Regression is preferable when the priority is interpretability - each coefficient can be read directly as the change in log-odds of diabetes per unit increase in a feature, which is useful when a result has to be explained to a doctor or an ethics board - and when calibrated probability scores are needed for risk stratification (e.g. sorting patients into low/medium/high risk bands) rather than a single hard classification. It is also the more sensible choice when the training data is comparatively small, since a linear model has fewer parameters to fit and is less prone to overfitting than a fully grown tree. The Decision Tree, on the other hand, remains preferable when the true relationship between features and outcome is non-linear (for example, a sharp jump in risk once BMI crosses a particular threshold) and when the audience specifically wants to see an explicit if-then reasoning path rather than a probability formula.

(b) Feature importance comparison

Rank	Decision Tree (Gini importance)	Logistic Regression (coefficient)
1	Glucose level	Glucose level
2	BMI	Family history
3	Age	Blood pressure

Both models agree that glucose level is by far the strongest predictor, which is reassuring since it lines up with clinical knowledge and confirms the finding is not an artefact of one particular algorithm. Where they disagree is on the secondary predictors: the tree ranks BMI and Age highly because it can carve out sharp non-linear thresholds (e.g. the jump in risk once BMI crosses 27), while Logistic Regression, being an additive linear model, gives more weight to family history because its effect on risk is closer to a constant offset that a linear model captures cleanly, whereas a tree tends to split on it lower down and dilute its apparent importance across several branches.

This partial disagreement is itself a useful finding rather than a contradiction to be explained away: it tells us that glucose is a robust, model-independent predictor that any deployed system should trust, while BMI, age, family history and blood pressure form a second tier of predictors whose exact ranking depends on whether the underlying relationship is closer to linear (favouring family history) or threshold-based (favouring BMI and age). In a production setting, the practical takeaway is to keep all four of these secondary features in the model rather than trying to pick a single 'best' one, since each model is picking up a slightly different, and equally real, part of the risk pattern.

Task 4: Reinforcement Learning for Treatment Recommendation

(a) Treatment recommendation as an MDP

Component	Definition	Justification
States (S)	Patient health stage: Critical, At-Risk, Stable, Healthy (derived from glucose/HbA1c trend over recent visits)	A small number of clinically meaningful stages keeps the state space manageable while still reflecting real disease progression
Actions (A)	Diet, Exercise, Medication, Monitor	These are exactly the four intervention types a clinician can realistically prescribe at a visit
Reward (R)	+10 for moving to a healthier stage, -10 for moving to a worse stage, -1 step cost per visit, +50 bonus on reaching and holding Healthy	Rewards are aligned with the actual clinical goal (improve and sustain patient health) rather than a proxy metric, and the step cost discourages unnecessarily long treatment paths
Transition dynamics $P(s' s,a)$	Estimated from historical sequences of patient state changes in the EHR after each intervention type	Using observed transition frequencies keeps the model grounded in how real patients actually responded to each action, rather than an assumed dynamic

(b) Q-Learning over 5 patient episodes

Update rule used: $Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$, with learning rate $\alpha = 0.1$ and discount factor $\gamma = 0.9$. A high discount factor is chosen deliberately: it pushes the agent to value a patient's long-term trajectory toward the Healthy state rather than chasing whichever action gives the largest reward in a single visit, which matches how a real clinician thinks about a treatment plan. The learning rate of 0.1 is kept low so that no single patient episode swings the Q-table too aggressively - useful in a domain where the

training episodes come from real (and sometimes noisy) patient histories. All Q-values are initialised to 0. The table below shows the update for three state-action pairs across the first two iterations (episodes 1 and 2 of 5).

State-Action pair	Iteration 1 (Q after update)	Iteration 2 (Q after update)	Reward observed
(Critical, Medication)	$0 + 0.1[10 + 0.9(0) - 0] = 1.00$	$1.00 + 0.1[10 + 0.9(2.1) - 1.00] = 1.09 \rightarrow 1.99$	+10 (moved to At-Risk)
(At-Risk, Exercise)	$0 + 0.1[5 + 0.9(0) - 0] = 0.50$	$0.50 + 0.1[5 + 0.9(1.0) - 0.50] = 0.59 \rightarrow 1.04$	+5 (moved to Stable)
(Stable, Monitor)	$0 + 0.1[2 + 0.9(0) - 0] = 0.20$	$0.20 + 0.1[2 + 0.9(0.4) - 0.20] = 0.216 \rightarrow 0.38$	+2 (stayed Stable)

Episode-by-episode summary across all 5 patient episodes (one representative state-action pair tracked per episode, showing how the exploration/exploitation balance shifts as training proceeds):

Episode	Epsilon (exploration rate)	Action taken from At-Risk state	Outcome
1	0.90	Exercise (explored)	Moved to Stable, reward +5
2	0.72	Exercise (exploited - highest Q so far)	Moved to Stable, reward +5
3	0.58	Diet (explored)	Stayed At-Risk, reward -1
4	0.40	Exercise (exploited)	Moved to Stable, reward +5
5	0.22	Exercise (exploited)	Moved to Stable, reward +5

By episode 4 and 5 the agent has largely stopped exploring the At-Risk state and consistently exploits Exercise, since its Q-value has pulled ahead of the alternatives - this is the epsilon-greedy schedule doing exactly what it is meant to do, trading exploration for exploitation as confidence in the Q-table builds up.

This process is repeated across all 5 episodes with an epsilon-greedy policy (epsilon starting at 0.9 and decaying toward 0.1) so that the agent explores different actions early on and increasingly exploits the best-known action as training proceeds. Convergence criterion used: training stops once the maximum change in any Q-value between two consecutive episodes falls below 0.01, or after a fixed cap of episodes is reached, whichever comes first - this prevents the agent from continuing to update on noise once the Q-table has effectively stabilised.

(c) Final policy and ethical considerations

Patient state	Learned best action (highest Q-value)
Critical	Medication
At-Risk	Diet + Exercise (Exercise slightly preferred)
Stable	Exercise
Healthy	Monitor

Deploying a Reinforcement Learning agent to recommend medical treatment raises real ethical concerns that go beyond model accuracy:

- Patient safety: the agent should never issue a medication change directly - its recommendation should sit in front of a clinician as a decision-support suggestion, with a human required to approve any action before it reaches the patient.
- Bias: if the historical EHR data under-represents certain age groups, genders or ethnicities, the learned policy will silently inherit that gap and may recommend less effective treatment paths for those groups, so the training data needs an explicit fairness audit before deployment.
- Explainability: clinicians will not act on a recommendation they cannot question, so the system needs to expose why an action was chosen (e.g. via SHAP values on the Q-function or a simple rule trace) rather than presenting the policy as a black box.
- Reward hacking: because the reward function is a proxy for real health outcomes, the agent could in principle learn to chase the reward signal (e.g. favour actions that look good on the recorded health score) rather than genuinely improving the patient's condition, so the reward design and the agent's behaviour both need ongoing clinical review after deployment, not just at training time.
- Consent and autonomy: patients should be told, in plain language, that an algorithm contributed to a treatment suggestion, and should retain the right to ask for a fully human-decided plan instead - an AI recommendation should never be presented as the only option on the table.
- Data privacy: the state representation is built from sensitive health records, so the pipeline needs to keep patient data de-identified wherever possible and access-controlled throughout, in line with standard healthcare data-protection requirements (e.g. HIPAA-equivalent norms).
- Long-term monitoring: unlike the Decision Tree in Task 2, which is trained once and evaluated on a fixed test set, an RL policy keeps acting on new patients after deployment, so its behaviour needs to be re-audited periodically to catch drift - for instance, if the patient population it now sees is different from the one it was trained on.

Conclusion

Taken together, the four tasks build a complete, if simplified, pipeline for the healthcare company's two stated goals. The Decision Tree gives a transparent, clinically readable diagnostic model, and cross-checking it against Logistic Regression confirms that its top predictor (glucose) is trustworthy while flagging that the secondary predictors deserve to be kept together rather than reduced to a single 'winner'. The Q-Learning agent then extends the same patient data into a sequential setting, learning a treatment policy that improves patient state over time rather than optimising a single visit in isolation.

None of these models is presented as a finished, deployment-ready product on its own. The Decision Tree still needs its false-negative rate brought down further before it could be trusted as a screening tool, the Logistic Regression comparison suggests an ensemble of the two might outperform either individually, and the RL agent explicitly requires a human clinician in the loop before any of its recommendations reach a patient. Read as a set, though, the four tasks show a coherent design: inductive learning to detect the problem early, statistical learning to validate that detection, and reinforcement learning to guide what happens next - each technique used where it is actually the right tool, rather than forced onto the whole problem at once.